

MongoDB

Experiment-06

06. Execute Aggregation Pipeline and its operations (pipeline must contain \$match, \$group, \$sort, \$project, \$skip etc. students encourage to execute several queries to demonstrate various aggregation operators).

Aggregation is a way of processing a large number of documents in a collection by means of passing them through different stages. The stages make up what is known as a pipeline. The stages in a pipeline can filter, sort, group, reshape and modify documents that pass through the pipeline.

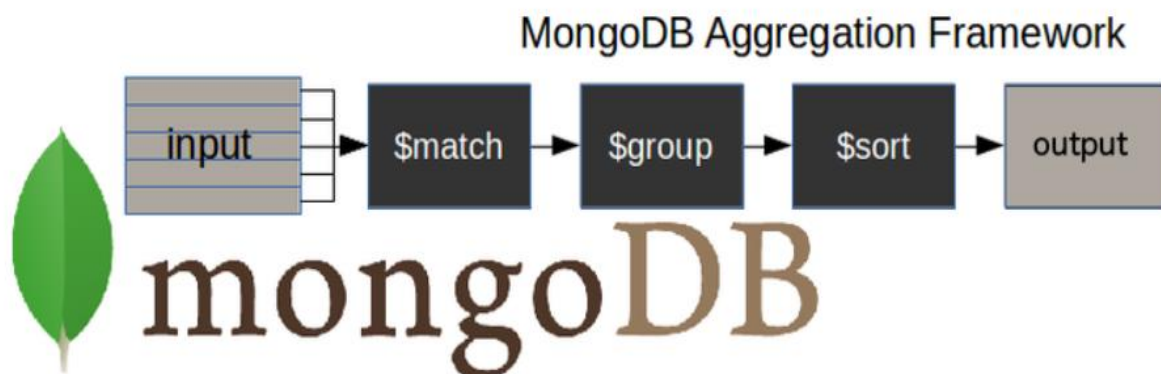
One of the most common use cases of Aggregation is to calculate aggregate values for groups of documents. This is **similar to the basic aggregation available in SQL with the GROUP BY clause and COUNT, SUM and AVG functions**. MongoDB Aggregation goes further though and can also perform relational-like joins, reshape documents, create new and update existing collections, and so on.

While there are other methods of obtaining aggregate data in MongoDB, the aggregation framework is the recommended approach for most work.

The **map-reduce framework** on MongoDB is a predecessor of the aggregation framework and much more complex to use.

How does the MongoDB aggregation pipeline work?

Here is a diagram to illustrate a typical MongoDB aggregation pipeline.



Aggregation Stage

- **\$match:** Filters documents based on a condition.
- **\$group:** Groups documents based on a specified field or expression, optionally performing calculations on the grouped data.
- **\$project:** Selects or transforms fields for the output documents.
- **\$unwind:** Deconstructs an array into separate documents for each element.
- **\$lookup:** Performs a join operation with another collection.
- **\$sort:** Sorts the output documents.
- **\$limit:** Limits the number of output documents.
- **\$skip:** Skips a specified number of documents from the beginning.

Aggregation Pipeline: Powerful Data Transformation in MongoDB

The aggregation pipeline in MongoDB is a versatile framework for efficiently transforming, grouping, and summarizing data. It allows you to perform complex data manipulations in a structured, stage-by-stage manner, offering several advantages over traditional find queries.

1. The input of the pipeline can be a single collection, where others can be merged later down the pipeline.
2. The pipeline then performs successive transformations on the data until our goal is achieved.
3. This way, we can break down a complex query into easier stages, in each of which we complete a different operation on the data. So, by the end of the query pipeline, we will have achieved all that we wanted.
4. This approach allows us to check whether our query is functioning properly at every stage by examining both its input and the output. The output of each stage will be the input of the next.
5. There is no limit to the number of stages used in the query, or how we combine them.
6. To achieve optimum query performance there are a number of best practices to take into account. We will come to those later in the article.

MongoDB aggregate pipeline syntax

This is an example of how to build an aggregation query:

`db.collectionName.aggregate(pipeline, options);`

- where *collectionName* – is the name of a collection.
- *pipeline* – is an array that contains the aggregation stages.
- *options* – optional parameters for the aggregation.

Key Stages

1. \$match:

Purpose: Filters documents based on specified criteria, reducing the amount of data processed in subsequent stages. This enhances performance and focuses the analysis on relevant data.

Example:

```
db.products.aggregate([
  { $match: { price: { $gt: 100 } } }, // Filter products with price > 100
])
```

Explanation: This pipeline first selects only products with a price greater than 100, ensuring subsequent operations work on a smaller, more targeted dataset.

Example 01:

This pipeline first filter students older than 23 then sort by age in descending order and projects only name and age.

```
db> db.student6.aggregate([ { $match: { age: { $gt: 23 } } },
...   { $sort: { age: -1 } },
...   { $project: { _id: 0, name: 1, age: 1 } } ] );
[ { name: 'Charlie', age: 28 }, { name: 'Alice', age: 25 } ]
db> |
```

Example 02:

This pipeline first filter students lesser than 23 then sort by age in ascending order and projects only name and scores.

```
db> db.student6.aggregate([ { $match: { age: { $lt: 23 } } }, { $sort: { name: 1 } }, { $project: { _id: 0, name: 1, scores: 1 } } ] );
[
  { name: 'Bob', scores: [ 90, 88, 95 ] },
  { name: 'David', scores: [ 98, 95, 87 ] }
]
db> |
```

2. \$group:

Purpose: Groups documents based on shared fields, allowing you to compute aggregate values like sum, average, count, or custom calculations for each group. It's ideal for summarizing data by categories or patterns.

Example:

```
db.orders.aggregate([
  { $group: { _id: "$category", totalSales: { $sum: "$amount" } } }, // Group
  by category and sum amounts
])
```

Explanation: This pipeline groups orders by their "category" field and calculates the total sales (totalSales) for each category using the \$sum accumulator.

Example 03:

This pipeline groups orders by their "major" field and calculates the averageAge for each major using the \$sum accumulator.

```
db> db.student6.aggregate([
... { $group: { _id: "$major", averageAge: { $avg: "$age" }, totalStudents: { $sum: 1 } } } ] );
[
  { _id: 'Mathematics', averageAge: 22, totalStudents: 1 },
  { _id: 'English', averageAge: 28, totalStudents: 1 },
  { _id: 'Biology', averageAge: 23, totalStudents: 1 },
  { _id: 'Computer Science', averageAge: 22.5, totalStudents: 2 }
]
db> |
```

Example 04:

This pipeline groups orders by their "name" field and calculates the averageAge for each name using the `\$sum` accumulator.

```
db> db.student6.aggregate([ { $group: { _id: "$name", averageAge: { $avg: "$age" }, totalStudents: { $sum: 1 } } }]);
[
  { _id: 'Alice', averageAge: 25, totalStudents: 1 },
  { _id: 'Bob', averageAge: 22, totalStudents: 1 },
  { _id: 'Charlie', averageAge: 28, totalStudents: 1 },
  { _id: 'David', averageAge: 20, totalStudents: 1 },
  { _id: 'Eve', averageAge: 23, totalStudents: 1 }
]
```

3. \$sort:

Purpose: Orders documents in a specified sequence (ascending or descending) based on one or more fields. Sorting is often necessary for presenting data in a meaningful way, especially after grouping.

Example:

```
db.students.aggregate([
// ... other pipeline stages
  { $sort: { averageGrade: -1 } } // Sort by averageGrade in descending order
])
```

Explanation:

This pipeline sorts students in descending order of their `averageGrade` (assuming it's already calculated), displaying the highest-performing students first.

Example 05: Sorting age by ascending order.

```
mongosh mongodb://127.0.0.1:27027/
db> db.student.aggregate([ { $sort: { age: 1 } } ] );
[
  {
    _id: ObjectId('665899c18fb275953df1283c'),
    name: 'Fatima Brown',
    age: 18,
    courses: [ 'Biology', 'Chemistry', 'Environmental Science' ],
    gpa: 3.5,
    home_city: 'San Antonio',
    blood_group: 'B+',
    is_hotel_resident: false
  },
  {
    _id: ObjectId('665899c18fb275953df12839'),
    name: 'Charlie Lee',
    age: 19,
    courses: [ 'History', 'English', 'Psychology' ],
    gpa: 3.2,
    home_city: 'Chicago',
    blood_group: 'B+',
    is_hotel_resident: true
  },
  {
    _id: ObjectId('665899c18fb275953df12840'),
    name: 'Jessica Moore',
    age: 19,
    courses: [ 'Biology', 'Ecology', 'Marine Science' ],
    gpa: 3.1,
    home_city: 'Austin',
  }
]
```

4.\$project:

Purpose: Selects and transforms output documents, allowing you to control which fields are included in the final results and potentially rename or modify them. It's useful for shaping the final dataset to meet your specific needs.

Example:

```
db.customers.aggregate([
  // ... other pipeline stages

  { $project: { _id: 0, name: 1, loyaltyPoints: { $multiply: ["$points", 2] } } }
// Exclude _id, project name, and multiply points by 2

])
```

Explanation: This pipeline excludes the `\_id` field, projects the `name` field, and creates a new field `loyaltyPoints` by doubling the existing `points` value.

Example 06: This pipeline excludes the `\_id` field, projects the name field and averagescore and matches the score older than 85 and skip 1.

```
db> db.student6.aggregate([
... {
... $project:{
... _id:0,
... name:1,
... averageScore:{ $avg: "$scores" },
... },
... },
... { $match: { averageScore: { $gt: 85 } } },
... { $skip: 1 }
... ]);
[ { name: 'David', averageScore: 93.33333333333333 } ]
db>
```

5. \$skip:

Purpose: Skips a specified number of documents at the beginning of the results. It's particularly helpful for pagination, allowing you to retrieve specific "pages" of data.

Example (assuming \$limit is used for pagination):

```
db.articles.aggregate([
    // ... other pipeline stages

    { $skip: (pageNumber - 1) * pageSize }, // Skip based on page number and
page size

    { $limit: pageSize } // Limit documents per page

])
```

Explanation: This pipeline retrieves data for a specific page. It calculates the number of documents to skip based on the current `pageNumber` and `pageSize`, then limits the output to the desired number of documents per page using `\$limit`.

Example 07: This pipeline excludes the `_id` field, projects the name field and averagescore and matches the score lesser than 86 and skip 2.

```
db> db.student6.aggregate([ { $project: { _id: 0, name: 1, averageScore: { $avg: "$scores" } } }, { $match: { averageScore: { $lt: 86 } } }, { $skip: 2 } ] );  
[ { name: 'Eve', averageScore: 83.33333333333333 } ]  
db> |
```

Example 08: This pipeline excludes the `_id` field, projects the name field and averagescore and matches the score lesser than 86 and skip 1.

```
db> db.student6.aggregate([ { $project: { _id: 0, name: 1, averageScore: { $avg: "$scores" } } }, { $match: { averageScore: { $lt: 86 } } }, { $skip: 1 } ] );  
[  
  { name: 'Charlie', averageScore: 82 },  
  { name: 'Eve', averageScore: 83.33333333333333 }  
]  
db> |
```

Advantages of Using Aggregation Pipeline:

- **Efficient processing:** The pipeline executes in stages, reducing unnecessary processing on filtered data.
- **Modular:** Break down complex transformations into smaller, more manageable stages.
- **Scalable:** Handles large datasets efficiently.
- **Flexible:** Offers a wide range of operators for various data manipulation needs.